

Aula 10 - Two Pointers

Two Pointers é uma técnica muito utilizada para a resolução de problemas de programação competitiva. A ideia consiste em usar dois ponteiros, l e r , que representam a posição mais a esquerda de um intervalo, o l , e a posição mais a direita, o r . Dados dois ponteiros, calculamos a resposta para esse intervalo e, baseado em alguma condição que varia de problema em problema, escolhemos o próximo intervalo a ser observado. Para isso, podemos incrementar ou decrementar algum dos dois ponteiros.

Como qualquer técnica de programação, a ideia fica mais sólida com a resolução de alguns problemas.

Problema: 2-Sum

Dado um vetor de inteiros v de tamanho n , e um inteiro X , queremos saber se existe ou não um par de inteiros $v[i]$ e $v[j]$ tal que $v[i] + v[j] = X$. OBS.: v está ordenado de forma crescente.

Ideia ingênua

Para cada elemento i , percorremos todos os outros elementos do vetor (de $i+1$ até $n-1$) testando se a soma é a desejada. Algo como:

```
bool check_2_sum(vector<int>& v, int X){ // aproveitando pa
ra lembrar como
// passar um vetor como parâmetro para uma função corretame
nte (&)
    int n = v.size();

    for(int i = 0; i<n-1; ++i){
        for(int j = i+1; j<n; ++j){
            if(v[i] + v[j] == X) return true;
        }
    }
}
```

```
    return false;
}
```

Dados os 2 loops aninhados, não é difícil enxergar que esse algoritmo é $O(n^2)$. Podemos melhorar isso.

Abordagem Two Pointers

A princípio, nosso intervalo é o vetor inteiro. Ou seja, vamos declarar nossos dois vetores `l` e `r` (left e right) como `l = 0` e `r = n-1`.

Daí, a nossa condição de atualização para o intervalo vai ser o seguinte teste:

- `v[l] + v[r]` é maior, menor, ou igual a `x`?
 - Se for maior → precisamos diminuir essa soma
 - Se for menor → precisamos aumentar essa soma
 - Se for exatamente igual → retornamos true e seremos felizes

Assim, só nos resta elaborar uma forma inteligente de “aumentar essa soma” e “diminuir essa soma”. Dado a natureza ordenada do vetor, sabemos que, para qualquer i , $v[i + 1] \geq v[i]$.

Então, para diminuir a soma, basta puxar o ponteiro `r` para a esquerda. E, para aumentar a soma, basta puxar `l` para a direita.

E, se chegar ao ponto de `r == l` e não conseguirmos nenhuma soma igual a X , é porque não temos nenhum par de inteiros em v que soma X . O código fica assim:

```
bool check_2_sum(vector<int>& v, int X){
    int l = 0, r = v.size()-1;

    while(l < r){
        if(v[l] + v[r] > X) r--;
        else if(v[l] + v[r] < X) l++;
        else return true;
    }

    return false;
}
```

Como só acessamos cada elemento do vetor uma única vez $\rightarrow O(n)$.

Problema: Soma de Intervalos Válida

Dado um vetor de inteiros v com tamanho n , e um inteiro X , queremos saber **quantos** intervalos (i, j) com $i < j$ existem tal que:

$$v[i] + v[i + 1] + \dots + v[j - 1] + v[j] = X$$

Ideia Ingênua

Para cada elemento i , iteramos pelos outros elementos (de $i+1$ até $n-1$) checando sempre as somas dos elementos nos intervalos entre i e j . Cada soma de intervalos é da ordem $O(n)$, sendo que isso é executado dentro de um loop aninhado, nos dando no final $O(n^3)$.

Ficaria mais ou menos assim:

```
for(int i = 0; i < n-1; ++i){
    for(int j = i+1; j < n; ++j){

        int sum = 0;
        for(int k = i; k <= j; ++k){
            sum += v[k];
        }

        if(sum == X) count++;
    }
}
```

Conseguimos melhorar (e muito!).

Abordagem com Two Pointers

Primeiro, declaramos nossos dois ponteiros $l = 0$ e $r = 0$ (sim, no começo, ambos são iguais e apontam pro começo do vetor). Daí, seguiremos um loop simples de decisões:

- Se a soma do intervalo (l, r) está menor que X , podemos incluir mais elementos nessa soma, logo caminhamos o ponteiro r para a direita.
- Se a soma do intervalo (l, r) está maior que X , precisamos retirar elementos dessa soma, logo caminhamos l para a direita.
- Se a soma do intervalo (l, r) é exatamente X , incrementamos um contador e caminhamos qualquer um dos ponteiros, vamos caminhar o l nesse caso.

Ficaria mais ou menos assim:

```
int sum = 0;
int l = 0, r = 0;

while(l < n){
    if(sum > X){
        l++;
        sum -= v[l-1];
    }
    else if(sum < X){
        r++;
        sum += v[r];
    }
    else{
        cont++;
        l++;
    }
}

cout << cont;
```

Como cada elemento é lido no máximo duas vezes nesse vetor, temos $O(2n) \rightarrow O(n)$ operações.

Esse estilo de two pointers é frequentemente chamado de **sliding window**.